

Appunti di Core Java 21

Emanuele Romano

Inizio percorso: 28 Marzo 2026

Indice

1	Fondamenta	1
1.1	Editing, Compilazione ed Esecuzione	1
1.2	Cenni alla JShell: L'approccio REPL	1
1.3	Tipi Primitivi	2
1.3.1	Il tipo char e Unicode	2
1.3.2	Il tipo boolean	3
1.3.3	Quadro generale dei tipi primitivi	3
1.4	Stringhe	4
1.4.1	L'Immutabilità in Java: Oltre le Stringhe	5
1.5	La Classe StringBuilder	5
1.6	Input e output su console	6
1.6.1	Scanner	6
1.6.2	Output su Console	6
1.7	Lettura e Scrittura su File	7
1.7.1	Lettura da File	7
1.7.2	Scrittura su File	8
1.7.3	Gestione dei Percorsi e Eccezioni	8
1.8	Il Costrutto Switch	8
1.8.1	Lo Switch Tradizionale (Statement)	8
1.8.2	La Sintassi a Freccia (Modern Switch Statement)	9
1.8.3	Switch Expressions	9
1.8.4	Evoluzioni Avanzate: Pattern Matching e Null	10
1.8.5	L'Utilizzo della Clausola <code>when</code> negli <code>switch</code>	10
1.9	Big Numbers: BigInteger e BigDecimal	11
2	Classi e oggetti	12
2.1	I Costruttori	12
2.1.1	Ulteriori note su <code>this</code>	13
2.2	Overloading e Firma dei Metodi	13
2.3	Costruzione degli Oggetti	14
2.3.1	Inizializzazione e Valori di Default	14
2.3.2	Blocchi di Inizializzazione	15
2.3.3	L'Ordine di Esecuzione	15
2.4	L'inferenza del tipo: la parola chiave <code>var</code>	15
2.5	I vantaggi dell'incapsulamento	16
2.5.1	Controllo, Flessibilità e Validazione	16
2.5.2	Il rischio della fuga di riferimenti (<i>Leaking References</i>)	16
2.6	I Metodi Privati	17
2.6.1	Helper Methods e Pulizia del Codice	17
2.6.2	Libertà di Evoluzione del Software	17
2.7	Campi di istanza <code>final</code>	17
2.7.1	Tipi primitivi e Classi Immutabili	17
2.7.2	La trappola delle Classi Mutabili	18

2.8	Membri Statici: Campi e Costanti	18
2.8.1	Campi Statici (Variabili di Classe)	18
2.8.2	I Metodi Statici	19
2.9	Il Metodo <code>main</code>	19
2.9.1	Novità Java 21: Instance Main Methods (Preview)	19
2.10	Il passaggio dei parametri: Call-by-Value	20
2.11	Tipi Enumerati (Enum)	20
2.12	I Records: classi come contenitori di dati	22
2.13	Gestione della Memoria: Stack vs Heap	24
2.13.1	Confronto Sintetico	25
A	Quick Reference: Metodi di uso frequente	26
A.1	Classe <code>String</code>	26
A.2	Classe <code>Math</code>	26
A.3	<code>BigInteger</code> e <code>BigDecimal</code>	26
A.4	Gestione del Tempo (<code>java.time</code>)	26
A.5	Parsing e Conversione	27

Fondamenta

1.1 Editing, Compilazione ed Esecuzione

Il processo di sviluppo in Java si fonda sulla distinzione tra codice sorgente e codice eseguibile, mediata dalla Java Virtual Machine (JVM).

Si definiscono tre stadi sequenziali per la messa in funzione di un software:

1. **Editing:** Scrittura del codice sorgente in un file di testo con estensione `.java`.
2. **Compilazione:** Traduzione del sorgente in *bytecode* (formato intermedio indipendente dall'architettura) tramite il comando `javac`. L'output è un file `.class`.
3. **Esecuzione:** Caricamento e interpretazione del bytecode da parte della JVM tramite il comando `java`.

Osservazione 1.1. La padronanza della riga di comando è essenziale per operare in ambienti server (sprovvisti di interfaccia grafica), per la gestione manuale del *Classpath* e per comprendere i meccanismi di risoluzione delle dipendenze che gli IDE tendono ad automatizzare e occultare.

A partire da Java 11, è stata introdotta la funzionalità definita dalla JEP 330, che permette di eseguire un programma contenuto in un singolo file sorgente senza la creazione esplicita di un file `.class`.

Listing 1.1: Esecuzione rapida senza compilazione manuale

```
1 # Sintassi valida da Java 11+
2 java NomeFile.java
```

In questo caso, il bytecode viene generato esclusivamente in memoria e rimosso al termine dell'esecuzione, rendendo Java efficace anche per la scrittura di script rapidi e prototipazione.

1.2 Cenni alla JShell: L'approccio REPL

Introdotta con Java 9, lo strumento `jshell` fornisce un ambiente interattivo per l'esplorazione del linguaggio.

JShell è un sistema *Read-Eval-Print Loop*. A differenza del ciclo standard (*edit-compile-run*), il REPL legge l'input dell'utente, valuta l'espressione, stampa il risultato e attende l'istruzione successiva.

Caratteristiche principali

- **Assenza di Boilerplate:** Non è necessario definire classi o il metodo `main` per eseguire istruzioni.
- **Variabili Implicite:** Le espressioni valutate senza assegnazione vengono memorizzate in variabili generate automaticamente (es. `$1`, `$2`).
- **Comandi di Controllo:** I comandi che iniziano con lo slash (es. `/list`, `/vars`, `/exit`) permettono di gestire lo stato della sessione.

Osservazione 1.2. L'utilità di JShell risiede nella riduzione dell'attrito cognitivo durante l'apprendimento. Permette di isolare il comportamento di specifiche API o costrutti

sintattici (come il comportamento dei tipi primitivi in condizioni di overflow) senza la distrazione della struttura formale del progetto.

Listing 1.2: Esempio di sessione interattiva

```
1 jshell> 2 + 2
2 $1 ==> 4
3
4 jshell> int x = $1 * 3
5 x ==> 12
6
7 jshell> /vars
8 |      int $1 = 4
9 |      int x = 12
```

1.3 Tipi Primitivi

1.3.1 Il tipo char e Unicode

A differenza del linguaggio C, dove il tipo `char` occupa solitamente 1 byte (ASCII), in Java il tipo `char` occupa **2 byte** (16 bit) e segue lo standard **UTF-16**.

Osservazione 1.3 (L'illusione dei 16 bit). Originariamente, Java era stato progettato con l'idea che 65.536 caratteri fossero sufficienti. Poiché lo standard Unicode è cresciuto oltre questo limite, Java ha dovuto adottare le **Surrogate Pairs**: alcuni simboli (emoji, caratteri esotici) vengono rappresentati usando **due char** consecutivi (4 byte totali).

Differenza chiave: Code Point vs Code Unit

- **Code Point**: Il valore numerico effettivo assegnato a un simbolo nello standard Unicode (es. U+1F4E9).
- **Code Unit**: L'unità di memoria minima in Java (16 bit).

Listing 1.3: Esempio di conteggio con caratteri supplementari

```
1 // Usiamo la codifica esadecimale per l'emoji del puzzle
2 // (che in UTF-16 richiede due unita': \uD83E e \uDDE9)
3 String s = "\uD83E\uDDE9";
4
5 System.out.println(s.length()); // Restituisce 2, non 1!
```

In definitiva, la funzione `length()` non restituisce il numero di simboli grafici "umani", ma il numero di **code units** (unità da 16 bit) necessarie a rappresentarli:

- **Caso Standard**: Per i caratteri comuni (Basic Multilingual Plane), 1 simbolo = 1 `char` (2 byte). In questo caso, la lunghezza coincide con il numero di caratteri.
- **Caso Esotico**: Per i caratteri supplementari (come le emoji o simboli matematici rari), 1 simbolo = 2 `char` (4 byte). In questo caso, la lunghezza risulta **doppia** rispetto al numero di simboli reali.

IMPORTANTE. Nonostante le stringhe siano logicamente sequenze di `char` a 2 byte (UTF-16), le versioni moderne di Java (a partire da Java 9 in poi) le memorizzano come sequenze di singoli byte se il contenuto lo permette, risparmiando spazio senza cambiare il comportamento del codice.

1.3.2 Il tipo boolean

Il tipo `boolean` esprime i valori di verità dell'algebra di Boole: `true` e `false`.

Osservazione 1.4 (Rappresentazione in memoria). Nonostante logicamente basti 1 bit, la dimensione effettiva non è definita rigorosamente dalla specifica JVM. Nella pratica:

- Una variabile singola occupa solitamente **1 byte** per motivi di allineamento della memoria e velocità di accesso della CPU.
- Non esiste alcuna conversione (casting) tra tipi numerici e il tipo `boolean`, a differenza di quanto avviene in linguaggio C.

Listing 1.4: Rigidità del tipo boolean

```

1 // Errore in Java (ma valido in C)
2 // int x = 1; if (x) { ... }
3
4 // Forma corretta in Java
5 int x = 1;
6 if (x != 0) {
7     System.out.println("Vero");
8 }

```

1.3.3 Quadro generale dei tipi primitivi

La seguente tabella riassume gli 8 tipi primitivi di Java, evidenziando le caratteristiche tecniche e le limitazioni algebriche discusse.

Tipo	Memoria	Intervallo / Specifica	Note di Rilievo
<code>byte</code>	1 byte	$[-2^7, 2^7 - 1]$	Uso limitato (stream di dati).
<code>short</code>	2 byte	$[-2^{15}, 2^{15} - 1]$	Ormai raramente utilizzato.
<code>int</code>	4 byte	$[-2^{31}, 2^{31} - 1]$	Tipo standard per gli interi.
<code>long</code>	8 byte	$[-2^{63}, 2^{63} - 1]$	Richiede suffisso L (es. 42L).
<code>float</code>	4 byte	IEEE 754 (6-7 cifre dec.)	Richiede suffisso F (es. 3.14F).
<code>double</code>	8 byte	IEEE 754 (15 cifre dec.)	Default per decimali; approssimato.
<code>char</code>	2 byte	Unità di codice UTF-16	16-bit; gestisce <i>surrogate pairs</i> .
<code>boolean</code>	~1 byte	{true, false}	Nessuna conversione da/verso <code>int</code> .

Tabella 1.1: Quadro sinottico dei tipi primitivi in Java.

IMPORTANTE. Promemoria:

- **Aritmetica Modulare:** L'overflow degli interi è silenzioso e ciclico ($max + 1 = min$).
- **Precisione Binaria:** Non usare mai `double` o `float` per calcoli monetari o di precisione assoluta (usare `BigDecimal`).
- **Immutabilità Logica:** I tipi primitivi passano sempre per **valore**, mai per riferimento.

1.4 Stringhe

In linguaggi come Python è fondamentale la distinzione tra tipi mutabili e immutabili, resa piuttosto evidente dal fatto che oggetti mutabili hanno spesso un equivalente immutabile (si pensi alla coppia `list/tuple`). In Java questo aspetto è meno immediato sintatticamente, ma altrettanto cruciale.

In particolare, gli oggetti **String** sono **immutabili**: una volta creati, il loro stato non può essere alterato. Ogni tentativo di modifica produce, di fatto, un nuovo oggetto con un riferimento diverso in memoria.

Apparentemente questa scelta progettuale potrebbe sembrare inefficiente sotto il profilo delle performance (si pensi a cicli iterativi che concatenano stringhe). In realtà, si tratta di una scelta strategica per bilanciare sicurezza, gestione della memoria e velocità. Ecco i quattro motivi principali:

1. String Pool (Efficienza di Memoria)

Le stringhe sono tra i dati più frequenti in un'applicazione. Grazie all'immutabilità, Java può utilizzare lo *String Pool*: una zona di memoria dove viene conservata una sola copia di ogni stringa letterale. Se definiamo `String a = "Ciao"` e `String b = "Ciao"`, entrambe le variabili punteranno allo **stesso identico oggetto**. Se le stringhe fossero mutabili, la modifica di `a` altererebbe inavvertitamente anche `b`. L'immutabilità permette quindi un enorme risparmio di RAM.

2. Sicurezza (Il motivo critico)

Le stringhe gestiscono dati sensibili come:

- Percorsi di file (es. `C:\dati\segreti.txt`);
- Credenziali di database (URL, username, password);
- Parametri di rete (Indirizzi IP e porte).

Se una stringa fosse mutabile, un thread malevolo potrebbe cambiare il contenuto del percorso di un file un istante dopo che il sistema di sicurezza ha autorizzato l'accesso, ma un istante prima dell'apertura effettiva. Questo bug è noto come **Time-of-Check to Time-of-Use (TOCTOU)**.

3. Caching dell'Hashcode (Performance)

[DA RIVEDERE DOPO AVER FATTO BENE HASHCODE E COLLECTIONS] Poiché il contenuto di una `String` non cambia mai, Java ne calcola l'hashcode una sola volta e lo memorizza in *cache*. Questo rende le stringhe estremamente veloci quando usate come chiavi nelle `HashMap` o nei `HashSet`. Se la stringa fosse mutabile, l'hash andrebbe ricalcolato ad ogni operazione di ricerca o inserimento.

4. Thread Safety

In contesti multi-core, gli oggetti immutabili sono *thread-safe* per definizione. Non è necessario implementare blocchi o meccanismi di sincronizzazione complessi (che rallentano l'esecuzione) per evitare che due thread modifichino la stessa stringa contemporaneamente, corrompendo i dati.

Osservazione 1.5. Nei casi in cui l'immutabilità comporti una reale inefficienza (come la costruzione di stringhe complesse in cicli intensivi), Java mette a disposizione la classe `StringBuilder`, che permette manipolazioni mutabili e che analizzeremo in un paragrafo a seguire.

1.4.1 L'Immutabilità in Java: Oltre le Stringhe

È un errore comune pensare che l'immutabilità riguardi esclusivamente la classe `String`. In realtà, Java adotta questo pattern per molte altre classi fondamentali, garantendo coerenza dei dati e facilità di debug. Un oggetto è definito immutabile se il suo stato non può essere alterato dopo la costruzione.

Oltre alle stringhe, i principali tipi immutabili in Java includono:

- **Classi Wrapper dei Primitivi:** Tutte le classi che "incapsulano" i tipi primitivi (`Integer`, `Long`, `Double`, `Float`, `Short`, `Byte`, `Character`, `Boolean`) sono immutabili. Se hai un `Integer` con valore 10, non puoi "sovrascrivere" quel 10; ogni operazione aritmetica produrrà un nuovo oggetto `Integer`.
- **Numeri a Precisione Arbitraria:** Le classi `BigInteger` e `BigDecimal` (utilizzate per calcoli matematici e finanziari di alta precisione) sono immutabili. Questo è cruciale perché garantisce che i valori calcolati non vengano alterati accidentalmente da altri moduli del programma.
- **Nuova API Date e Time (`java.time`):** A partire da Java 8, le classi per la gestione di date e orari (come `LocalDate`, `LocalTime`, `ZonedDateTime`) sono state rese immutabili. Questo ha risolto i gravi problemi di sicurezza e thread-safety della vecchia classe `java.util.Date`, che essendo mutabile era soggetta a manipolazioni impreviste.

Osservazione 1.6. L'immutabilità funge da **invariante logica**: in un sistema complesso, sapere che certi oggetti non cambieranno mai il proprio stato permette all'informatico di ragionare sul codice con la stessa certezza con cui si ragiona su una costante in una formula.

1.5 La Classe `StringBuilder`

Come analizzato nella paragrafo precedente, l'immutabilità delle stringhe garantisce sicurezza e risparmio di memoria tramite lo *String Pool*. Tuttavia, questa caratteristica diventa un limite critico in termini di performance quando è necessario modificare ripetutamente un testo (ad esempio all'interno di un ciclo).

La classe `StringBuilder` (definita in `java.lang`) è progettata per gestire stringhe ****mutabili****. Internamente gestisce un array di caratteri (un buffer) che può essere modificato direttamente senza creare nuovi oggetti.

Osservazione 1.7 (Capacity vs Length). `StringBuilder` distingue tra:

- **Length:** Il numero di caratteri effettivamente contenuti.
- **Capacity:** La dimensione del buffer interno. Se la lunghezza supera la capacità, l'array viene automaticamente ridimensionato (solitamente raddoppiato), garantendo un'efficienza ammortizzata.

Listing 1.5: Esempio di utilizzo efficiente in un ciclo

```
1 StringBuilder sb = new StringBuilder();
2 for (int i = 0; i < 100; i++) {
3     sb.append("Elemento ").append(i).append("; ");
4 }
5 String risultato = sb.toString(); // Conversione finale in String
```

A differenza di `String`, questa classe offre metodi per la modifica *in-place*:

- `append(valore)`: Aggiunge qualsiasi tipo di dato alla fine della sequenza.
- `insert(offset, valore)`: Inserisce testo in un punto specifico spostando i caratteri successivi.
- `delete(start, end)`: Rimuove una porzione di testo.
- `reverse()`: Inverte l'ordine dei caratteri (operazione estremamente efficiente).

IMPORTANTE. Ottimizzazione del Compilatore: Nelle versioni moderne di Java, il compilatore trasforma automaticamente le concatenazioni semplici (es. `s = a + b`) in istruzioni `StringBuilder`. Tuttavia, all'interno dei **cicli**, il compilatore non è in grado di applicare questa ottimizzazione in modo efficiente, rendendo l'uso manuale di `StringBuilder` un requisito fondamentale per scrivere codice professionale. Non conviene invece usarlo se devono solo unire due o tre stringhe una sola volta: l'overhead per istanziare l'oggetto `StringBuilder` renderebbe il codice inutilmente complesso e leggermente più lento rispetto ad una semplice concatenazione con l'operatore `+`.

1.6 Input e output su console

1.6.1 Scanner

Per leggere l'input dalla console (standard input), Java mette a disposizione la classe `Scanner`, definita nel pacchetto `java.util`.

Per utilizzare lo `Scanner`, bisogna creare un'istanza collegata allo stream `System.in`:

Listing 1.6: Inizializzazione dello Scanner

```
1 import java.util.Scanner;  
2  
3 Scanner in = new Scanner(System.in);
```

I metodi principali per acquisire dati sono:

- `nextLine()`: legge un'intera riga di testo (inclusi gli spazi).
- `next()`: legge una singola parola (si ferma al primo spazio bianco).
- `nextInt()`, `nextDouble()`: leggono rispettivamente un intero o un numero decimale.

Osservazione 1.8. A differenza del C, dove la gestione dell'input con `scanf` può essere complessa e rischiosa per la sicurezza (buffer overflow), la classe `Scanner` gestisce internamente l'allocazione della memoria, rendendo l'acquisizione dei dati più astratta e sicura.

1.6.2 Output su Console

In Java, l'output standard viene gestito principalmente attraverso l'oggetto `System.out`. Esistono tre metodi fondamentali per inviare dati alla console:

- `print()`: Stampa il contenuto senza aggiungere un ritorno a capo alla fine.
- `println()`: Stampa il contenuto e aggiunge automaticamente un carattere di *new-line* (`\n`). È il metodo più utilizzato per il debug rapido.
- `printf()`: Introdotto per la compatibilità con lo stile del linguaggio C, permette la stampa di testo formattato tramite l'uso di *placeholder* (specificatori di formato).

Simbolo	Tipo di dato
%d	Intero decimale
%s	Stringa
%f	Numero a virgola mobile
%n	Ritorno a capo (piattaforma-indipendente)

Lo specificatore printf

La sintassi di `printf` segue lo schema: `System.out.printf(formato, argomenti)`. Ecco i simboli più comuni:

Listing 1.7: Esempio di output formattato

```
1 double quota = 1000.0 / 3.0;
2 // Stampa con 2 cifre decimali e un totale di 8 spazi di
  larghezza
3 System.out.printf("%8.2f", quota);
```

Osservazione 1.9. Mentre in C potevi accidentalmente passare un argomento del tipo sbagliato a `printf` causando crash o comportamenti indefiniti, in Java il compilatore e la JVM effettuano controlli più rigorosi, lanciando un'eccezione (`IllegalFormatException`) se il tipo dell'argomento non corrisponde allo specificatore.

IMPORTANTE. Curiosità: Se hai bisogno di formattare una stringa senza stamparla immediatamente, puoi usare `String.format("...", args)`, che restituisce una nuova stringa formattata seguendo le stesse regole di `printf`.

1.7 Lettura e Scrittura su File

Per operare con i file, Java estende l'uso della classe `Scanner` per la lettura e introduce la classe `PrintWriter` per la scrittura. Entrambe richiedono la gestione dei percorsi tramite la classe `Path`.

1.7.1 Lettura da File

Per leggere un file, non è sufficiente passare una stringa con il nome del file allo `Scanner`. È necessario creare un oggetto `Path`.

Listing 1.8: Lettura corretta di un file

```
1 import java.nio.file.Path;
2 import java.util.Scanner;
3 import java.nio.charset.StandardCharsets;
4
5 // Creazione del percorso e apertura dello scanner
6 Scanner in = new Scanner(Path.of("miofile.txt"), StandardCharsets
  .UTF_8);
```

ATTENZIONE. Se scrivi `Scanner in = new Scanner("miofile.txt")`, lo `Scanner` non aprirà il file, ma leggerà la stringa stessa come se fosse il contenuto dei dati. L'uso di `Path.of` è obbligatorio per puntare al file fisico.

1.7.2 Scrittura su File

Per scrivere testo su un file, si utilizza la classe `PrintWriter`. Se il file non esiste, verrà creato; se esiste, il contenuto verrà sovrascritto.

Listing 1.9: Scrittura su file

```
1 import java.io.PrintWriter;
2
3 PrintWriter out = new PrintWriter("output.txt", StandardCharsets.
    UTF_8);
4 out.println("Risultato del calcolo: " + risultato);
5 out.close(); // Fondamentale per svuotare il buffer e salvare
```

1.7.3 Gestione dei Percorsi e Eccezioni

Quando si lavora con i file, bisogna considerare due aspetti critici:

- **Percorsi:** Puoi usare percorsi relativi o assoluti (es. `C:\\Users\\test.txt`). Ricorda che nei percorsi Windows inseriti nel codice Java il backslash va raddoppiato.
- **Eccezioni:** L'accesso ai file può fallire (file mancante, permessi negati). Per ora, Java ti obbliga a dichiarare che il metodo `main` può generare un errore aggiungendo `throws IOException` alla firma del metodo.

IMPORTANTE. Evoluzione del Charset (JEP 400): Sebbene a partire da Java 18 l'encoding predefinito sia diventato ufficialmente `UTF_8`, è comunque consigliabile specificare `StandardCharsets.UTF_8` (spesso considerato ridondante) in modo esplicito. Questo garantisce la portabilità del codice su versioni precedenti di Java e rende l'intento del programmatore inequivocabile, evitando che il comportamento del software dipenda implicitamente dalle impostazioni della JVM.

1.8 Il Costrutto Switch

Lo `switch` permette di selezionare un ramo di esecuzione basandosi sul valore di un'espressione. Java 21 ha trasformato radicalmente questo costrutto.

1.8.1 Lo Switch Tradizionale (Statement)

La forma classica (stile C) utilizza i due punti (`:`) e richiede il `break` per evitare il *fall-through*.

Listing 1.10: Switch tradizionale

```
1 switch (scelta) {
2     case 1:
3         System.out.println("Uno");
4         break;
5     case 2:
6         System.out.println("Due");
7         break;
8     default:
9         System.out.println("Altro");
10 }
```

1.8.2 La Sintassi a Freccia (Modern Switch Statement)

Introdotta per risolvere l'intrinseca fragilità del **break**, questa sintassi utilizza l'operatore **->**. In questa forma, lo **switch** è ancora uno **statement** (esegue un'azione ma non restituisce un valore).

- **Niente break:** Viene eseguito solo il codice a destra della freccia. Non c'è rischio di cadere nel caso successivo.
- **Casi multipli:** Si possono raggruppare più valori con la virgola: `case 1, 2, 3 ->`
....

Listing 1.11: Switch moderno con freccia

```
1 switch (scelta) {  
2     case 1 -> System.out.println("Uno");  
3     case 2 -> System.out.println("Due");  
4     case 3, 4 -> {  
5         // Se serve piu di una riga, si usano le graffe  
6         System.out.println("Tre o Quattro");  
7     }  
8     default -> System.out.println("Altro");  
9 }
```

1.8.3 Switch Expressions

Quando lo **switch** viene usato per assegnare un valore a una variabile, diventa una **expression**. In questo caso, il costrutto **deve** essere esaustivo (coprire tutti i casi o avere un default).

Listing 1.12: Switch come espressione

```
1 String etichetta = switch (scelta) {  
2     case 1 -> "Uno";  
3     case 2 -> "Due";  
4     default -> "Altro";  
5 }; // Nota il punto e virgola finale, obbligatorio nelle  
    espressioni
```

L'istruzione yield

Se un ramo della *switch expression* richiede un blocco logico complesso, si usa **yield** per "emettere" il valore finale del blocco.

Listing 1.13: Uso di yield

```
1 int calcolo = switch (modo) {  
2     case 1 -> 10;  
3     default -> {  
4         int temp = eseguiCalcolo();  
5         yield temp * 2;  
6     }  
7 };
```

1.8.4 Evoluzioni Avanzate: Pattern Matching e Null

Solo una volta compresa la sintassi a freccia, possiamo sfruttare la potenza di Java 21 per testare i **tipi** degli oggetti e gestire i valori `null`.

Listing 1.14: Pattern Matching e gestione null

```
1 switch (obj) {  
2     case null      -> System.out.println("Nessun oggetto");  
3     case Integer i -> System.out.println("Intero: " + i);  
4     case String s  -> System.out.println("Stringa: " + s);  
5     default       -> System.out.println("Tipo non supportato");  
6 }
```

Osservazione 1.10. Sebbene esistano diverse combinazioni sintattiche per lo `switch` (incrociando l'uso di `:` o `->` con la natura di *statement* o *expression*), la buona pratica in Java moderno è orientata verso la massima sicurezza. Si tende a utilizzare quasi esclusivamente la **sintassi a freccia** (`->`) per eliminare alla radice il rischio di *fall-through* e a preferire, laddove possibile, le **switch expressions** per rendere il codice più dichiarativo e meno incline a errori di stato.

1.8.5 L'Utilizzo della Clausola `when` negli `switch`

La clausola **when** (o *Guards*) estende la potenza dell'istruzione `switch` permettendo di aggiungere condizioni booleane specifiche a ogni `case`.

Invece di limitarsi a confrontare un valore costante, **when** permette di filtrare il match in base a variabili dinamiche.

Listing 1.15: Esempio di Pattern Matching con clausola `when`

```
1 switch (dispositivo) {  
2     case Smartphone s when s.getLivelloBatteria() < 10 ->  
3         pulisciMemoria();  
4     case Smartphone s when s.isModalitaAereo() ->  
5         disabilitaRete();  
6     default ->  
7         attendiSegnale();  
8 }
```

L'introduzione di questa clausola è particolarmente vantaggiosa in tre scenari principali:

- **Rimozione dei rami condizionali annidati:** Evita di dover inserire blocchi `if-else` all'interno di un `case`, rendendo il codice più lineare e leggibile (cosiddetto *Flattening*).
- **Pattern Matching Avanzato:** Permette di gestire lo stesso tipo di oggetto in modi diversi nello stesso `switch`. Ad esempio, posso avere due `case` per la classe `Ordine`, uno `when total > 1000` e uno per i restanti.
- **Espressività Semantica:** Sposta la logica di controllo direttamente nella firma del caso, rendendo immediatamente chiaro sotto quali condizioni quel ramo viene eseguito.

1.9 Big Numbers: BigInteger e BigDecimal

Quando la precisione dei tipi primitivi interi (`long`) o a virgola mobile (`double`) non è sufficiente, il pacchetto `java.math` mette a disposizione due classi fondamentali: **BigInteger** e **BigDecimal**. Queste classi permettono di manipolare numeri con una sequenza di cifre arbitrariamente lunga.

Esistono tre modi principali per creare un "Big Number":

- **Metodo statico `valueOf`**: Ideale per convertire numeri ordinari.

```
1 BigInteger a = BigInteger.valueOf(100);
```

- **Costruttore con `String`**: Indispensabile per numeri che superano la capacità dei tipi primitivi.

```
1 BigInteger reallyBig = new BigInteger("
    2222322446294204455297398934619099672066669390964000099");
```

- **Costanti predefinite**: `BigInteger.ZERO`, `ONE`, `TWO` e `TEN`.

IMPORTANTE. Per `BigDecimal`, si deve **sempre** usare il costruttore che accetta una `String`. Il costruttore `BigDecimal(double)` è intrinsecamente pronò a errori di arrotondamento dovuti alla rappresentazione binaria dei floating-point.

Ad esempio, `new BigDecimal(0.1)` non produce esattamente 0.1, ma una sequenza imprecisa (com'è noto da nozioni di architettura dei calcolatori, 0.1 in binario è un numero periodico infinito, quindi viene approssimato):

0.10000000000000000055511151231257827021181583404541015625

L'uso della stringa "0.1" garantisce invece la precisione esatta richiesta.

Poiché Java non supporta l'overloading degli operatori (a differenza di C++), non è possibile usare `+`, `-`, `*` o `/`. È necessario utilizzare i metodi d'istanza dedicati.

Operazione	Metodo Java
Somma (+)	<code>a.add(b)</code>
Sottrazione (−)	<code>a.subtract(b)</code>
Moltiplicazione (*)	<code>a.multiply(b)</code>
Divisione (/)	<code>a.divide(b)</code>
Modulo (%)	<code>a.mod(b)</code>

Tabella 1.2: Metodi aritmetici per `BigInteger` e `BigDecimal`.

Osservazione 1.11 (Immutabilità). Proprio come le `String`, anche i Big Numbers sono **immutabili**. Metodi come `add` non modificano l'oggetto originale nello **Heap**, ma restituiscono una nuova istanza con il risultato.

Classi e oggetti

2.1 I Costruttori

Un costruttore è un metodo speciale che viene invocato esclusivamente tramite l'operatore `new`. Esso rappresenta il blocco di codice che viene eseguito nel momento esatto in cui un nuovo oggetto di una classe viene istanziato.

- **Scopo:** L'obiettivo primario di un costruttore è impostare lo stato iniziale dell'oggetto, ovvero inizializzare i suoi campi di istanza affinché l'oggetto nasca in uno stato valido.
- **Nome e Ritorno:** Per convenzione del linguaggio, il costruttore deve avere lo stesso identico nome della classe e **non deve specificare alcun tipo di ritorno** (nemmeno `void`).

Listing 2.1: Esempio di costruttore base

```
1 public class Employee {
2     private String name;
3     private double salary;
4
5     // Costruttore
6     public Employee(String n, double s) {
7         name = n;
8         salary = s;
9     }
10 }
```

Osservazione 2.1. È importante ricordare che un costruttore non può essere invocato su un oggetto già esistente per "re-inizializzarlo". Viene eseguito una sola volta per ogni istanza, all'atto della creazione.

Spesso, per rendere il codice più leggibile ed evitare la proliferazione di identificatori, si tende a dare ai parametri del costruttore lo stesso nome dei campi di istanza della classe. In questi casi, il nome del parametro "nasconde" (*shadowing*) il campo di istanza. Per risolvere l'ambiguità e indicare esplicitamente che ci si riferisce alla variabile dell'oggetto e non al parametro locale, si utilizza la parola chiave `this`.

In Java, `this` è un riferimento all'oggetto corrente su cui il metodo o il costruttore è stato invocato. Scrivere `this.nomeCampo` indica in modo univoco al compilatore di accedere alla variabile memorizzata nello *heap* per quell'istanza specifica.

Listing 2.2: Uso di `this` nel costruttore

```
1 public class Employee {
2     private String name;
3     private double salary;
4
5     public Employee(String name, double salary) {
6         // this.name si riferisce al campo istanza
7         // name si riferisce al parametro del costruttore
8         this.name = name;
```

```

9         this.salary = salary;
10    }
11 }

```

2.1.1 Ulteriori note su `this`

Oltre alla necessità tecnica di risolvere conflitti di nomi, molti programmatori utilizzano `this` all'interno dei metodi e dei costruttori anche laddove non sarebbe strettamente necessario (ovvero quando i nomi dei parametri sono diversi da quelli dei campi).

Questa pratica ha un valore puramente **visivo e documentale**: anteporre `this.` permette di distinguere immediatamente, a colpo d'occhio, quali variabili appartengono allo stato permanente dell'oggetto e quali sono invece variabili locali o parametri temporanei. In progetti di grandi dimensioni, questa convenzione aiuta a ridurre il carico cognitivo di chi legge il codice e riduce drasticamente il rischio di errori accidentali durante la manutenzione dello stesso.

In Java, il riferimento `this` punta sempre all'oggetto corrente (il cosiddetto *parametro implicito*). Il suo utilizzo si divide in tre casi principali:

- **`this.nomeCampo`**: fondamentale, come si è visto poco sopra, per risolvere lo *shadowing*, ovvero quando un parametro del metodo o del costruttore ha lo stesso nome di un campo della classe.
- **`this.nomeMetodo(parametri)`**: utilizzato per invocare un altro metodo sulla stessa istanza.
- **`this(parametri)`**: Questa forma speciale permette a un costruttore di richiamarne un altro all'interno della stessa classe. Come vedremo in paragrafi a seguire, è la tecnica principe per gestire l'**overloading dei costruttori** senza duplicare il codice di inizializzazione. *Nota bene: deve essere obbligatoriamente la prima istruzione del costruttore.*

2.2 Overloading e Firma dei Metodi

L'overloading consente di definire più costruttori o metodi con lo stesso nome, purché abbiano parametri differenti.

- **Firma (Signature)**: È l'identità del metodo, composta da **nome + tipi dei parametri**.
- **Esclusione del Ritorno**: Il tipo di ritorno **non** fa parte della firma. Non è possibile avere due metodi con parametri identici ma tipi di ritorno diversi.

In questo esempio, la classe `Employee` gestisce tre campi: `name`, `salary` e `hireDay`. Utilizziamo l'overloading per offrire diversi gradi di flessibilità nella creazione dell'oggetto, delegando sempre l'inizializzazione al costruttore più completo.

Listing 2.3: Overloading a cascata nella classe `Employee`

```

1 public class Employee {
2     private String name;
3     private double salary;
4     private LocalDate hireDay;
5
6     // 1. Costruttore "Master" (3 parametri)
7     // E' l'unico che contiene la logica di assegnazione
    effettiva.

```



```
8      public Employee(String name, double salary, LocalDate hireDay
9          ) {
10          this.name = name;
11          this.salary = salary;
12          this.hireDay = hireDay;
13      }
14
15      // 2. Costruttore con 2 parametri
16      // Imposta la data di assunzione a "oggi" per default.
17      public Employee(String name, double salary) {
18          this(name, salary, LocalDate.now());
19      }
20
21      // 3. Costruttore con 1 parametro
22      // Imposta lo stipendio a 0 e la data a "oggi" per default.
23      public Employee(String name) {
24          this(name, 0, LocalDate.now());
25      }
26
27      // 4. Costruttore vuoto (no-arg)
28      // Imposta valori generici di default.
29      public Employee() {
30          this("Sconosciuto", 0, LocalDate.now());
31      }
32  }
```

Osservazione 2.2. Il vantaggio della manutenzione. Se in futuro si volesse aggiungere un controllo di validità (ad esempio, impedire che lo stipendio sia negativo), basterebbe inserire la logica nel **costruttore master**. Tutti gli altri costruttori ne beneficerebbero automaticamente, garantendo la coerenza dello stato dell'oggetto indipendentemente da come viene creato.

IMPORTANTE. Ricorda che la chiamata `this(...)` deve essere tassativamente la **prima riga** del costruttore. Non puoi eseguire alcun calcolo o istruzione prima di delegare la costruzione all'altro costruttore.

2.3 Costruzione degli Oggetti

Java offre diversi meccanismi per inizializzare lo stato di un oggetto. Per evitare confusione, è utile osservare il processo come una sequenza di regole logiche.

2.3.1 Inizializzazione e Valori di Default

A differenza delle variabili locali (che devono essere inizializzate esplicitamente), i campi di istanza vengono sempre azzerati automaticamente se non specificato diversamente:

- Numeri (int, double, ecc.): 0.
- Booleani: false.
- Riferimenti a oggetti: null.

Attenzione 2.1. Il costruttore predefinito "gratis". Java fornisce implicitamente un costruttore senza argomenti (*no-arg constructor*) solo se nella classe non è stato definito **nessun** costruttore. Se ne definisci anche solo uno con parametri, quello predefinito scompare; se ti serve ancora, devi scriverlo esplicitamente.

2.3.2 Blocchi di Inizializzazione

Oltre all'assegnazione diretta (nelle dichiarazioni dei campi di istanza) e ai costruttori, Java mette a disposizione un terzo meccanismo per inizializzare i campi: i **blocchi di inizializzazione**. Si tratta di segmenti di codice racchiusi tra parentesi graffe inseriti direttamente nel corpo della classe.

Esistono due tipi di blocchi, con scopi e tempi di esecuzione differenti:

- **Blocchi di istanza:** Vengono eseguiti ogni volta che viene creata una nuova istanza della classe, subito dopo l'inizializzazione dei campi ai valori predefiniti e prima del corpo del costruttore.
- **Blocchi statici (`static { ... }`):** Vengono eseguiti una sola volta, nel momento in cui la classe viene caricata in memoria dalla JVM. Sono ideali per configurazioni complesse di variabili statiche (ad esempio, caricare dati da un file o inizializzare un generatore di numeri casuali).

Osservazione 2.3. Sebbene potenti, i blocchi di inizializzazione di istanza sono raramente necessari e possono rendere il codice meno leggibile. La pratica consigliata è quella di inserire la logica di inizializzazione direttamente nei costruttori o di usare la concatenazione tramite `this(...)`.

2.3.3 L'Ordine di Esecuzione

Quando viene invocato l'operatore `new`, Java segue un ordine tassativo:

1. Tutti i campi sono azzerati (valori di default).
2. Vengono eseguiti gli eventuali inizializzatori espliciti (es. `private int id = 1;`) e gli eventuali blocchi di inizializzazione (`{...}`) nell'ordine in cui appaiono nel file.
3. Viene infine eseguito il corpo del costruttore selezionato.

2.4 L'inferenza del tipo: la parola chiave `var`

A partire da Java 10, il linguaggio ha introdotto la parola chiave `var`, un meccanismo noto come *local variable type inference*. Per un programmatore abituato al rigore del C, questo potrebbe sembrare un passaggio verso la tipizzazione dinamica, ma la realtà è differente: Java rimane un linguaggio a tipizzazione statica.

Il senso del `var` è eliminare la ridondanza visiva. Spesso, in Java, il nome del tipo appare sia nella dichiarazione che nell'inizializzazione (es. `Employee e = new Employee()`). Poiché il compilatore è in grado di dedurre il tipo osservando l'espressione alla destra dell'operatore di assegnazione, il `var` permette di omettere la dichiarazione esplicita del tipo senza perdere in sicurezza.

Listing 2.4: Confronto tra dichiarazione esplicita e `var`

```
1 // Approccio tradizionale
2 Employee harry = new Employee("Harry Hacker", 50000);
3
4 // Approccio con inferenza del tipo
5 var staff = new Employee("Harry Hacker", 50000);
```

Esistono tuttavia dei limiti tecnici e stilistici precisi per l'utilizzo di questo strumento:

- **Ambito locale:** Può essere utilizzato esclusivamente per le variabili locali all'interno dei metodi. Non è ammesso per i campi di istanza di una classe, dove il tipo deve essere sempre esplicito.

- **Inizializzazione immediata:** Una variabile dichiarata con `var` deve essere inizializzata nello stesso momento in cui viene dichiarata, altrimenti il compilatore non avrebbe elementi per dedurne il tipo.
- **Leggibilità:** Horstmann suggerisce di usare `var` solo quando il tipo è ovvio dalla lettura del lato destro. Ad esempio, `var in = new Scanner(System.in)` è chiaro, mentre `var x = calcola()` potrebbe rendere il codice criptico se non si conosce il valore di ritorno del metodo.

IMPORTANTE. È fondamentale ricordare che il tipo viene fissato a tempo di compilazione. Una volta che `var staff` è stato interpretato come `Employee`, non potrà mai contenere un tipo diverso (come una stringa o un intero) durante l'esecuzione del programma.

2.5 I vantaggi dell'incapsulamento

L'incapsulamento non consiste solo nel nascondere i dati, ma nel fornire un'interfaccia controllata per interagire con essi. Horstmann identifica tre benefici fondamentali nell'uso di metodi accessori (getter) e mutatori (setter) rispetto ai campi pubblici.

2.5.1 Controllo, Flessibilità e Validazione

1. **Debugging semplificato:** Se un campo viene modificato solo tramite un mutatore, la ricerca di eventuali bug legati a valori errati è circoscritta a quel metodo specifico.
2. **Trasparenza dell'implementazione:** La struttura interna dei dati può evolvere (es. dividere un campo `name` in `firstName` e `lastName`) senza che il codice esterno debba essere modificato, poiché l'interfaccia dei metodi rimane costante.
3. **Integrità dei dati:** I metodi mutatori possono eseguire controlli di validità (es. impedire stipendi negativi) prima di aggiornare lo stato interno.

2.5.2 Il rischio della fuga di riferimenti (*Leaking References*)

Un errore di progettazione sottile ma grave consiste nello scrivere metodi accessori che restituiscono riferimenti a oggetti **mutabili**.

Attenzione 2.2. Riferimenti mutabili e rottura dell'incapsulamento.

Se un campo privato è un oggetto mutabile (come la vecchia classe `Date`), restituire quel riferimento tramite un getter permette al codice esterno di modificare lo stato interno dell'oggetto all'insaputa della classe stessa.

Si consideri il seguente esempio di “codice canaglia”:

Listing 2.5: Violazione dell'incapsulamento

```
1 Employee harry = ...;
2 Date d = harry.getHireDay(); // d punta all'oggetto interno di
   harry
3 double tenYearsInMilliseconds = 10 * 365.25 * 24 * 60 * 60 *
   1000;
4 d.setTime(d.getTime() - (long) tenYearsInMilliseconds);
5 // Lo stato di harry è cambiato senza usare i metodi di Employee!
```

IMPORTANTE. Per prevenire questa vulnerabilità, esistono due strategie principali:

- **Utilizzare oggetti immutabili:** Classi come `LocalDate` o `String` non possiedono metodi mutatori. Una volta creati, il loro stato non può cambiare, rendendo sicuro il ritorno del loro riferimento.
- **Clonazione:** Se si deve assolutamente usare un oggetto mutabile, il getter dovrebbe restituire una copia (*clone*) dell'oggetto, non l'originale. Vedremo più avanti in dettaglio questo tipo di soluzione.

2.6 I Metodi Privati

Nella grande maggioranza dei casi, i metodi di una classe vengono dichiarati con il modificatore `public`, poiché costituiscono l'interfaccia attraverso cui il mondo esterno interagisce con l'oggetto. Tuttavia, esistono circostanze in cui è preferibile o necessario rendere un metodo `private`.

2.6.1 Helper Methods e Pulizia del Codice

Spesso, l'implementazione di un compito complesso richiede di spezzare il codice in diverse sotto-operazioni. Questi "metodi di supporto" (*helper methods*) non dovrebbero far parte dell'interfaccia pubblica: potrebbero essere troppo legati all'attuale implementazione interna o richiedere un protocollo di chiamata specifico che non deve essere esposto all'utente della classe. Dichiarandoli privati, ci si assicura che vengano utilizzati solo internamente.

2.6.2 Libertà di Evoluzione del Software

Il vantaggio principale della riservatezza dei metodi risiede nella libertà di manutenzione.

- **Metodi Pubblici:** Rappresentano un impegno verso chi utilizza la classe. Se un metodo è pubblico, non può essere rimosso o modificato drasticamente senza rischiare di "rompere" il codice esterno che vi fa affidamento.
- **Metodi Privati:** Finché un metodo rimane privato, il progettista della classe ha la certezza che non venga utilizzato altrove. Questo permette di rinominarlo, ottimizzarlo o addirittura eliminarlo completamente in futuro, qualora la rappresentazione interna dei dati dovesse cambiare, senza alcun impatto sul resto del programma.

Osservazione 2.4. L'uso di metodi privati è una forma di incapsulamento del comportamento. Proprio come i campi privati proteggono lo stato dell'oggetto da manipolazioni esterne, i metodi privati proteggono la logica di implementazione, garantendo che l'utente della classe veda solo ciò che è strettamente necessario al corretto funzionamento del sistema.

2.7 Campi di istanza `final`

In Java, è possibile dichiarare un campo di istanza come `final`. Tale modificatore impone che la variabile venga inizializzata tassativamente entro la fine di ogni costruttore della classe. Una volta assegnato, il valore del campo non può più essere modificato.

2.7.1 Tipi primitivi e Classi Immutabili

L'uso di `final` è particolarmente efficace con i tipi primitivi o con classi immutabili (ovvero classi i cui metodi non modificano mai lo stato dell'oggetto, come `String` o `LocalDate`). In questi casi, il campo diventa a tutti gli effetti una costante dopo la costruzione dell'oggetto.

2.7.2 La trappola delle Classi Mutabili

È necessario prestare attenzione quando si usa `final` con oggetti mutabili.

Attenzione 2.3. Il modificatore `final` garantisce solo che il **riferimento** memorizzato nella variabile non punti mai a un oggetto diverso. Tuttavia, non impedisce la mutazione dell'oggetto stesso: per i mutabili `final` è solo il riferimento all'oggetto ad essere costante, ma tutto ciò a cui punta quel riferimento può essere cambiato.

Ad esempio, un campo `private final StringBuilder evaluations` impedisce di assegnare un nuovo `StringBuilder` alla variabile, ma permette comunque di invocare metodi come `evaluations.append()`, modificando di fatto lo stato interno dell'oggetto.

Osservazione 2.5. Un campo `final` può essere inizializzato con il valore `null` (magari a seguito di un controllo condizionale). In tal caso, la variabile rimarrà `null` per l'intero ciclo di vita dell'istanza e non potrà mai essere riassegnata a un oggetto reale.

2.8 Membri Statici: Campi e Costanti

Il modificatore `static` permette di definire membri che appartengono alla **classe** stessa piuttosto che alle singole istanze.

2.8.1 Campi Statici (Variabili di Classe)

Se un campo è dichiarato `static`, ne esiste un'unica copia condivisa da tutti gli oggetti della classe. Anche se non viene istanziato alcun oggetto, il campo statico è presente in memoria.

Esempio 2.1. Nella classe `Employee`, un campo `private int id` è un campo di istanza (ogni dipendente ha il suo), mentre un campo `private static int nextId` è un campo di classe (condiviso da tutti per generare ID univoci).

Listing 2.6: Utilizzo di un campo statico nel costruttore

```
1 public Employee(...) {  
2     id = nextId; // Assegna l'ID corrente  
3     nextId++;   // Incrementa il contatore condiviso nella  
4                 classe  
5 }
```

Mentre le variabili statiche sono rare (e vanno usate con cautela!) le costanti statiche (`static final`) sono molto comuni. Esse permettono di accedere a valori fissi tramite il nome della classe, senza dover istanziare oggetti.

Osservazione 2.6. È buona norma accedere ai membri statici usando il nome della classe (es. `Employee.nextId`) invece che tramite un riferimento a un oggetto (es. `harry.nextId`), per rendere subito chiaro a chi legge che il campo non appartiene all'istanza.

2.8.2 I Metodi Statici

Qui il concetto è del tutto analogo a quello dei campi di istanza: si tratta di metodi comuni a tutta la classe. A differenza dei metodi di istanza, un metodo statico non ha un parametro implicito `this`; in altre parole, non riceve alcun riferimento a un'istanza specifica della classe quando viene invocato.

Poiché non dispongono del riferimento `this`, i metodi statici presentano caratteristiche precise:

- **Accesso ai dati:** un metodo statico può accedere ai campi statici della classe, ma non può mai accedere ai campi di istanza. Non operando su un oggetto, il metodo non “saprebbe” a quale istanza riferirsi per leggere i dati non statici.
- **Invocazione:** come per i campi, sebbene il linguaggio permetta di invocare un metodo statico tramite il riferimento a un oggetto, la buona pratica prevede di utilizzare sempre il nome della classe. Questo chiarisce immediatamente che l'operazione non dipende dallo stato di un singolo oggetto.

È appropriato definire un metodo come `static` quando il metodo deve solo eseguire un calcolo basato su parametri forniti esplicitamente (come `Math.pow(x, a)`), oppure quando il metodo deve accedere esclusivamente a campi statici della propria classe (come un metodo che incrementa un contatore globale di istanze).

2.9 Il Metodo `main`

Il metodo `main` è il punto di ingresso di ogni applicazione Java. La sua natura riflette la necessità della Java Virtual Machine (JVM) di avviare l'esecuzione prima ancora che vengano creati gli oggetti.

Il metodo `main` deve essere dichiarato `static` perché deve poter essere invocato dalla JVM senza che sia necessaria un'istanza della classe. Quando il programma parte, non esiste ancora alcun oggetto: il `main` statico viene eseguito, “accende il motore” e inizia a creare gli oggetti necessari all'applicazione.

Una caratteristica potente di Java è che **ogni classe** può contenere un metodo `main`. Questo non significa che l'applicazione avrà più punti di ingresso, ma permette di inserire del codice di test o di dimostrazione direttamente all'interno della classe stessa.

- Se si lancia la classe specifica (es. `java Employee`), viene eseguito il suo `main` (utile per testare se i metodi funzionano correttamente).
- Se la classe è usata all'interno di un'applicazione più grande, il suo `main` viene semplicemente ignorato.

2.9.1 Novità Java 21: Instance Main Methods (Preview)

A partire da Java 21, è stata introdotta una *preview feature* che semplifica l'avvio dei programmi. In questa nuova modalità:

- Il `main` non deve più essere obbligatoriamente `static` o `public`.
- Può non avere il parametro `String[] args`.

Se il `main` non è statico, la JVM crea automaticamente un'istanza della classe (usando il costruttore predefinito) e invoca il metodo su di essa. Questa evoluzione punta a ridurre il “boilerplate code” per i principianti e per piccoli script.

Osservazione 2.7. Per un programmatore C, il `main` di Java è l'equivalente della funzione globale `main()`, ma incapsulato in una classe per rispettare il paradigma ad oggetti. La versione “non statica” di Java 21 avvicina ancora di più il linguaggio alla semplicità dei linguaggi di scripting.

2.10 Il passaggio dei parametri: Call-by-Value

Esiste spesso confusione sulla modalità con cui Java passa i parametri ai metodi. La regola ferrea del linguaggio è una sola: **Java utilizza esclusivamente il passaggio per (copia del) valore (call-by-value)**. Ciò significa che il metodo riceve sempre una copia dei valori contenuti nelle variabili fornite come argomenti. Copie che, di fatto, sono quindi **variabili locali**.

Il comportamento osservato differisce in base a ciò che la variabile contiene:

- **Tipi Primitivi:** La variabile contiene il valore numerico o booleano. Il metodo riceve una copia di tale valore; ogni modifica effettuata all'interno del metodo rimane locale e non influenza la variabile originale.
- **Riferimenti a Oggetti:** La variabile contiene l'indirizzo di memoria dell'oggetto. Il metodo riceve una copia di questo indirizzo. Poiché sia la variabile originale che la copia puntano allo stesso oggetto, è possibile modificarne lo stato interno, ma **non è possibile** far sì che la variabile originale punti a un nuovo oggetto. In questo caso si parla di *passaggio per copia del valore del riferimento*.

IMPORTANTE. Il fallimento dello swap: La dimostrazione definitiva che Java non usa il passaggio per riferimento è l'impossibilità di scrivere un metodo `swap(x, y)` che scambi i riferimenti di due oggetti esterni. Il metodo scambierà solo le sue copie locali, lasciando invariate le variabili originali.

Osservazione 2.8. Il caso degli oggetti immutabili. Sebbene per le classi immutabili (come `String`, `Integer`, `LocalDate`) valga la regola del passaggio per copia del valore del riferimento, il loro comportamento pratico è identico a quello dei tipi primitivi. Poiché l'oggetto non può essere modificato internamente, qualsiasi operazione di "modifica" effettuata nel metodo (es. `s = s + "!"`) non è altro che una riassegnazione della variabile locale a un nuovo oggetto, lasciando il riferimento originale del chiamante del tutto invariato.

Osservazione 2.9. Per un programmatore C, questo sistema è identico al passaggio di un puntatore per valore: si può manipolare il dato puntato, ma non si può modificare l'indirizzo di memoria memorizzato nel puntatore originale del chiamante.

2.11 Tipi Enumerati (Enum)

In Java, un **Enum** non è una semplice lista di costanti intere (come in C o C++), ma una classe speciale che estende implicitamente la classe `java.lang.Enum`. Rappresenta un insieme fisso di costanti tipizzate, garantendo che una variabile di quel tipo possa assumere solo uno dei valori predefiniti.

L'utilizzo degli **enum** è considerato una *best practice* per diversi motivi:

- **Type Safety:** Impedisce il passaggio di valori non validi (evitando l'uso di "Magic Numbers" o costanti `int`).
- **Namespace:** Le costanti sono confinate nel tipo enumerato, evitando conflitti di nomi.
- **Potenzialità di Classe:** Possono avere campi, costruttori e metodi, permettendo di associare dati e comportamenti a ogni costante.
- **Immutabilità:** Le istanze di un enum sono create all'avvio e non possono essere modificate, rendendole intrinsecamente *thread-safe*.

Di seguito è riportato lo schema più completo per definire un **enum**, includendo attributi, costruttori e metodi personalizzati.

Listing 2.7: Sintassi generale di un Enum in Java

```

1 public enum NomeEnum {
2     // 1. Definizione delle costanti (devono apparire per prime)
3     COSTANTE_A("Descrizione A", 10),
4     COSTANTE_B("Descrizione B", 20),
5     COSTANTE_C; // Se non ci sono argomenti, il costruttore è di
6                 // default
7
8     // 2. Campi di istanza (solitamente private final per
9     // immutabilità)
10    private final String descrizione;
11    private final int valore;
12
13    // 3. Costruttore (sempre private o package-private)
14    private NomeEnum(String descrizione, int valore) {
15        this.descrizione = descrizione;
16        this.valore = valore;
17    }
18
19    // Costruttore di default (opzionale se necessario)
20    private NomeEnum() {
21        this.descrizione = "Default";
22        this.valore = 0;
23    }
24
25    // 4. Metodi (Getter, Logica di business, Override)
26    public String getDescrizione() {
27        return descrizione;
28    }
29
30    public int getValore() {
31        return valore;
32    }
33
34    @Override
35    public String toString() {
36        return String.format("%s (Codice: %d)", descrizione,
37                               valore);
38    }
39 }

```

Un `enum` viene utilizzato come un qualsiasi altro tipo di dato, con la differenza fondamentale che non può essere istanziato tramite l'operatore `new`. L'accesso alle costanti avviene tramite il nome della classe enumerata.

Esempio di integrazione in una Classe

Listing 2.8: Esempio di utilizzo di un Enum in una classe di business

```

1 public class Ordine {
2     private int id;
3     private StatoOrdine stato; // Riferimento all'enum

```



```

4
5     public Ordine(int id) {
6         this.id = id;
7         this.stato = StatoOrdine.NUOVO; // Valore iniziale
8     }
9
10    public void avanzaStato() {
11        // Esempio di switch moderno (Java 21+)
12        this.stato = switch (this.stato) {
13            case NUOVO -> StatoOrdine.IN_ELABORAZIONE;
14            case IN_ELABORAZIONE -> StatoOrdine.SPEDITO;
15            case SPEDITO -> StatoOrdine.CONSEGNATO;
16            case CONSEGNATO -> StatoOrdine.CONSEGNATO;
17        };
18    }
19 }

```

Ogni enum eredita automaticamente dei metodi statici molto potenti:

- `values()`: Restituisce un array contenente tutte le costanti dell'enum nell'ordine in cui sono dichiarate. Utile per iterazioni.
- `valueOf(String name)`: Converte una stringa nel corrispondente valore enum. Lancia `IllegalArgumentException` se la stringa non corrisponde a nessuna costante.
- `ordinal()`: Restituisce la posizione intera (base 0) della costante nella dichiarazione. *Nota: se ne sconsiglia l'uso per logiche di business persistenti poiché fragile rispetto a cambiamenti dell'ordine.*

Osservazione 2.10 (Confronto e Memoria). Poiché le istanze di un **enum** sono **singleton** (ne esiste una sola copia nello **Heap** per tutta la durata dell'applicazione), il confronto tra due variabili enum può essere effettuato in modo sicuro ed efficiente utilizzando l'operatore `==` invece di `.equals()`. Questo non solo è più leggibile, ma evita anche potenziali `NullPointerException`.

2.12 I Records: classi come contenitori di dati

A partire da Java 16, il linguaggio introduce i **Records**, una forma specializzata di classe progettata per modellare aggregati di dati immutabili in modo conciso. I records eliminano il cosiddetto *boilerplate code* tipico delle classi Java tradizionali (POJO - Plain Old Java Objects).

Un record si definisce dichiarandone i campi, in gergo chiamati **componenti** del record, tra parentesi tonde:

Listing 2.9: Dichiarazione di un Record

```

1 public record Point(double x, double y) { }

```

Questa singola riga di codice equivale a una classe completa che include:

- Campi privati e finali: `private final double x;`
- Un costruttore che inizializza i campi.
- Metodi accessori (getters): `p.x()` e `p.y()` (nota l'assenza del prefisso `get`).
- Implementazioni standard di `toString()`, `equals()` e `hashCode()`.

Le caratteristiche principali dei record sono:

- **Immutabilità:** I record sono progettati per essere "viste" immutabili di dati. Una volta creato, lo stato di un record non dovrebbe cambiare.
- **Estensibilità:** Un record può implementare interfacce, definire metodi di istanza, metodi statici e costruttori personalizzati, ma **non può** aggiungere altri campi di istanza né estendere altre classi.
- **Personalizzazione:** È possibile sovrascrivere i metodi generati automaticamente, ad esempio per aggiungere una logica di validazione nel costruttore.

Attenzione 2.4. Attenzione alla mutabilità dei componenti. I componenti di un record sono implicitamente *final*, quindi va ricordato ciò che abbiamo detto a proposito di tali campi: se un record ha come componente un oggetto mutabile (es. un array), l'immutabilità del record è solo superficiale; il riferimento all'array non può cambiare, ma gli elementi interni all'array sì. Per garantire la sicurezza, è preferibile usare solo tipi primitivi o immutabili come componenti.

Osservazione 2.11. Analogia con il C: Per un programmatore C, il record è l'equivalente funzionale di una `struct` costante, ma con il vantaggio di avere già pronti tutti i metodi per il confronto e la visualizzazione testuale.

Un esempio base di record:

Listing 2.10: Dichiarazione ed istanza di un Record

```

1 // Dichiarazione del record (genera automaticamente campi,
2 // costruttore, getter, equals, hashCode e toString)
3 public record Persona(String nome, int eta) {}
4
5 // Utilizzo
6 public class Main {
7     public static void main(String[] args) {
8         // Creazione di un'istanza
9         Persona p = new Persona("Mario Rossi", 30);
10
11         // Accesso ai dati (notare l'assenza di "get")
12         System.out.println(p.nome());
13     }
14 }
```

In questo esempio personalizziamo il comportamento interno del record:

Listing 2.11: Record con logica personalizzata

```

1 public record Prodotto(String codice, double prezzo) {
2
3     // 1. Override del costruttore (Compact Constructor)
4     // Usato per validazione o normalizzazione
5     public Prodotto {
6         if (prezzo < 0) {
7             throw new IllegalArgumentException("Il prezzo non puo
8                 essere negativo");
9         }
10     }
11 }
```

```
9         codice = codice.toUpperCase(); // Normalizzazione
10     }
11
12     // 2. Metodo proprio (aggiuntivo)
13     public double prezzoConIva() {
14         return prezzo * 1.22;
15     }
16
17     // 3. Override di un Getter (accessor)
18     @Override
19     public String codice() {
20         return "ID-" + codice;
21     }
22
23     // 4. Override di toString
24     @Override
25     public String toString() {
26         return "Prodotto: " + codice + " costa " + prezzo + "
27             Euro";
28     }
29 }
```

Tip: Quando usare un Record?

Usa un **record** invece di una classe tradizionale quando l'oggetto è completamente definito dal suo stato (i suoi attributi) e tali attributi sono **costanti** e si è sicuri di non doverne aggiungere altri in futuro.

È la scelta ideale per DTO (Data Transfer Objects), chiavi di mappe o semplici contenitori di dati, a patto di poter accettare le restrizioni dei record (niente ereditarietà, campi final).

2.13 Gestione della Memoria: Stack vs Heap

Un obiettivo fondamentale per comprendere il comportamento di Java, specialmente durante l'analisi delle classi e degli oggetti, è la distinzione tra l'allocazione nello **Stack** e nello **Heap**.

Lo Stack è una sorta di memoria di un metodo in esecuzione, una sezione di memoria rapida e organizzata in modo lineare (LIFO).

- **Contenuto:** Memorizza le variabili locali e i **tipi primitivi** (int, double, boolean, ecc.).
- **Riferimenti:** Quando viene creato un oggetto, lo Stack non contiene l'oggetto stesso, ma solo l'indirizzo di memoria (riferimento) necessario per rintracciarlo nello Heap.
- **Ciclo di vita:** La memoria viene gestita automaticamente; quando un metodo termina l'esecuzione, il suo spazio nello Stack viene rimosso immediatamente.

Lo Heap è un'area di memoria molto più vasta e dinamica.

- **Contenuto:** Qui risiedono tutti i veri **oggetti** (istanze di classi) e gli **array**. Anche le stringhe letterali vengono conservate qui, all'interno del cosiddetto *String Pool*.

- **Accesso:** Gli oggetti nello Heap sono accessibili solo tramite i riferimenti memorizzati nello Stack.
- **Ciclo di vita:** Gli oggetti rimangono nello Heap finché sono raggiungibili da almeno un riferimento. Quando non sono più referenziati, il **Garbage Collector** si occupa di deallocarli per liberare spazio.

Osservazione 2.12. Comprendere questa distinzione chiarisce il comportamento dei parametri nei metodi:

1. **Passaggio per copia del valore (primitivi):** Viene copiata la variabile sullo Stack; le modifiche nel metodo non influenzano l'originale.
2. **Passaggio per copia del valore del riferimento (oggetti):** Viene copiato l'indirizzo memorizzato nello Stack. Poiché entrambi i riferimenti puntano allo **stesso oggetto nello Heap**, le modifiche apportate all'oggetto sono visibili globalmente.

2.13.1 Confronto Sintetico

Caratteristica	Stack	Heap
Tipo di accesso	Sequenziale (LIFO)	Casuale
Velocità	Molto Elevata	Più Lenta
Cosa ospita	Primitivi e Riferimenti	Oggetti, Array, String Pool
Gestione	Automatica (Compiler)	Garbage Collector
Visibilità	Privata al Thread	Condivisa (rischio thread-safety)

Tabella 2.1: Differenze chiave tra Stack e Heap in Java.

IMPORTANTE. Connessione con la Concorrenza: In sistemi complessi o distribuiti, ogni thread possiede il proprio Stack privato, mentre lo Heap è condiviso tra tutti. Questa condivisione è la causa principale dei problemi di visibilità e sincronizzazione che richiedono l'uso di blocchi (**synchronized**) o variabili atomiche.

Quick Reference: Metodi di uso frequente

In questa appendice sono raccolti i metodi delle librerie standard di Java 21 necessari per risolvere la maggior parte degli esercizi algoritmici senza consultare la documentazione esterna.

A.1 Classe String

Le stringhe sono immutabili. Ogni metodo restituisce una nuova istanza.

Metodo	Descrizione
<code>length()</code>	Restituisce il numero di caratteri.
<code>charAt(int i)</code>	Carattere alla posizione i .
<code>substring(start, end)</code>	Sottostringa dall'indice <i>start</i> a <i>end-1</i> .
<code>contains(CharSequence)</code>	Verifica se la sequenza è presente.
<code>split(String regex)</code>	Divide la stringa in un array di sottostringhe.
<code>toLowerCase()</code> / <code>toUpperCase()</code>	Cambio di <i>case</i> .
<code>strip()</code> / <code>trim()</code>	Rimuove spazi bianchi (<code>strip</code> è più moderno/Unicode).

A.2 Classe Math

Metodi statici per operazioni matematiche comuni.

Metodo	Esempio / Risultato
<code>Math.min(a, b)</code> / <code>Math.max(a, b)</code>	Restituisce il minore o il maggiore.
<code>Math.abs(x)</code>	Valore assoluto $ x $.
<code>Math.sqrt(x)</code>	Radice quadrata $\sqrt{x}$.
<code>Math.pow(base, esp)</code>	Elevamento a potenza (restituisce <code>double</code>).
<code>Math.round(x)</code>	Arrotondamento all'intero più vicino.

A.3 BigInteger e BigDecimal

Metodi per l'aritmetica ad alta precisione.

Operazione	Sintassi Java
Somma / Sottrazione	<code>a.add(b)</code> / <code>a.subtract(b)</code>
Moltiplicazione / Divisione	<code>a.multiply(b)</code> / <code>a.divide(b, roundingMode)</code>
Confronto	<code>a.compareTo(b)</code> (0 se uguali, 1 se $a > b$, -1 se $a < b$)

A.4 Gestione del Tempo (java.time)

Ricorda: queste classi sono immutabili. Ogni modifica crea un nuovo oggetto.

Metodo	Descrizione / Esempio
<code>LocalDate.now()</code>	Data corrente del sistema.
<code>LocalDate.of(y, m, d)</code>	Crea una data specifica (es. 2026, 4, 2).
<code>date.plusDays(n)</code>	Aggiunge n giorni (restituisce nuovo oggetto).
<code>date.isBefore(other)</code>	Verifica se la data precede un'altra.
<code>ChronoUnit.DAYS.between(a, b)</code>	Calcola i giorni tra due date.

A.5 Parsing e Conversione

Metodi per trasformare stringhe in tipi di dato e viceversa.

Operazione	Sintassi Java
Da String a int	<code>Integer.parseInt("123")</code>
Da String a double	<code>Double.parseDouble("12.5")</code>
Da int/double a String	<code>String.valueOf(n)</code>
Format di output	<code>System.out.printf("%d - %.2f", i, d)</code>